Supabase

Supabase를 활용한 Vibe Web Coding

Supabase란?

Supabase는 PostgreSQL 데이터베이스를 기반으로 웹 및 모바일 애플리케이션의 백엔드 기능을 제공하는 오픈소스 BaaS(Backend as a Service) 플랫폼입니다.
일반적인 웹 개발에서는 데이터베이스, 서버, 사용자 인증, API 등을 개발자가 직접 구성해야 하지만, Supabase를 이용하면 이러한 기능을 상당 부분 직접 구현하지 않고 사용할 수 있습니다.

일반적인 웹 개발과 Supabase 비교

구분	일반적인 웹 개발	Supabase 활용
프론트엔드	HTML, CSS, JavaScript	HTML, CSS, JavaScript
백엔드	FastAPI, Node.js 등	Supabase가 주요 기능 제공
데이터베이스	MySQL, PostgreSQL 등	PostgreSQL 기본 제공
API 개발	직접 구현	REST API 자동 생성
사용자 인증	직접 개발	Supabase Auth
파일 저장	별도 서버 또는 스토리지	Supabase Storage
데이터 접근 권한	서버에서 직접 구현	RLS 정책 활용
서버 운영	직접 구성 및 관리	클라우드 서비스 이용
배포	웹 서버 구성	GitHub Pages 등과 연동

Supabase

PostgreSQL 기반의
오픈소스 백엔드 플랫폼

- 백엔드 개발 부담을 줄이고
- 데이터베이스, 인증, 저장소, 실시간 기능까지
- 웹·모바일 앱을 빠르게 만들 수 있는 올인원 플랫폼

웹 · 모바일 앱 (클라이언트)

HTML, CSS, JavaScript 등으로
만든 프론트엔드에서
Supabase와 연동하여 사용



- 정적 웹사이트 (예: GitHub Pages)
- React, Vue 등 웹 앱
- 모바일 앱 (iOS / Android)

Supabase
JavaScript SDK

HTTPS
(안전한 연결)

1. PostgreSQL 데이터베이스

- 강력한 관계형 데이터베이스
- 테이블, 관계, 제약조건
- SQL, View, Function, Trigger
- 트랜잭션 지원

2. 자동 생성 REST API

API

- 테이블 기반 API 자동 제공
- CRUD 기능 바로 사용
- 복잡한 서버 개발 없이 데이터 접근 가능

supabase

PostgreSQL

3. 사용자 인증 (Auth)

- 이메일/비밀번호 회원가입
- 로그인/로그아웃
- 소셜 로그인 (Google 등)
- 비밀번호 재설정
- 사용자 정보 관리

4. RLS (Row Level Security)

- 행 단위 데이터 접근 권한 제어
- 사용자별 데이터 접근 제한
- 브라우저에서 안전하게 사용 가능

5. 실시간 기능 (Realtime)

- 데이터 변경 시 실시간 반영
- 채팅, 알림, 공동 작업 등에 활용

6. 피일 저장 (Storage)

- 이미지, 동영상, 문서 등 저장
- 공개/비공개 접근 권한 설정
- 웹에서 바로 업로드 및 다운로드

7. Edge Functions

- 서버 측 로직 실행 (JavaScript / TypeScript)
- 외부 API 호출, 결제 연동, 이메일 발송 등
- 비밀 키를 사용하는 보안 작업 처리

8. SQL Editor

- 브라우저에서 직접 SQL 실행
- 테이블 생성, 데이터 관리
- RLS 정책, Function, Trigger 설정

개발 속도 향상
백엔드 개발 없이
빠른 서비스 구축

무료 플랜 제공
학습 및 쇼규모
프로젝트에 적합

오픈소스
직접 호스팅 가능

다양한 플랫폼 지원
웹, 모바일, 다양한
프레임워크와 연동 가능


마당 서점
신간도서
베스트셀러
마당 서점
책이 있는
행복한 마당
마당
마당 서점과
함께하는
즐거운 독서생활

마당 서점 도서 구매 Process

기능	사용 방법
도서 목록	누구나 볼 수 있습니다.
도서 검색	제목으로 검색합니다.
회원가입	이메일과 비밀번호를 등록합니다.
로그인	가입한 계정으로 로그인합니다.
도서 구매	로그인 후 수량을 선택하여 모의 주문합니다.
내 주문	자신의 주문만 확인합니다.

모의주문 : 실제 결제는 이루어지지 않음

시스템 구축을 위한 작업 순서

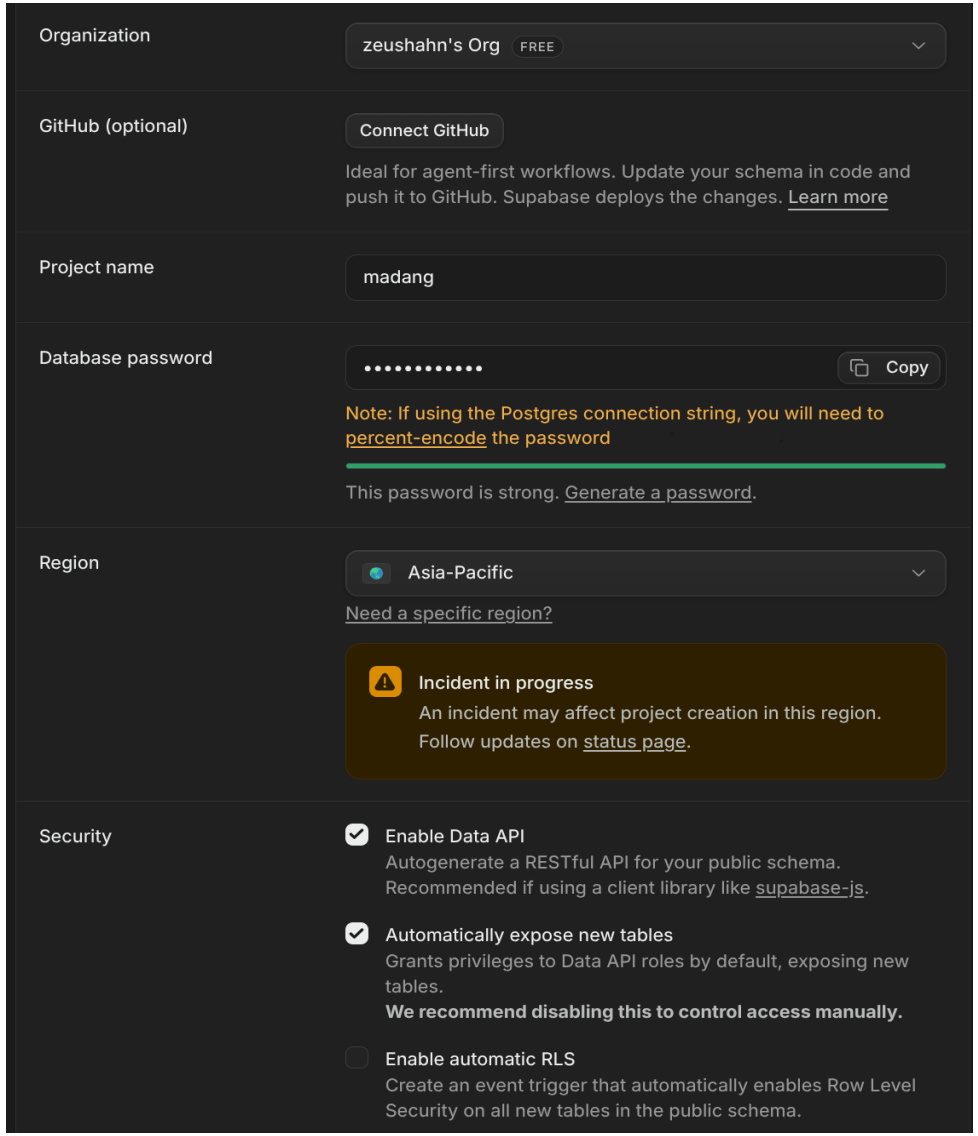
1. Supabase 프로젝트 만들기
2. Codex로 DB 구성 SQL 만들기
3. Supabase에 SQL 실행하기
4. Codex로 웹페이지 만들기
5. 회원가입·구매 기능 연결하기
6. GitHub Pages로 배포하기

1. Supabase 프로젝트 만들기 : 프로젝트 만들기

- https://supabase.com/ 에 접속
- Start your project를 선택
- GitHub 계정으로 로그인
- 조직 생성 화면이 나오면 개인 조직 생성
- 프로젝트 만들기

① New project를 선택합니다.
② 다음 항목을 입력합니다.
③ DB 비밀번호를 개인적으로 보관합니다.
④ Create new project를 선택합니다.
⑤ 프로젝트 준비가 끝나면 프로젝트 화면으로 들어갑니다.

항목	입력 내용
Organization	본인의 조직
Project name	`madang`
Database password	생성하거나 직접 설정한 긴 비밀번호(보관필요)
Region	서울이 있으면 서울, 없으면 가까운 지역



1. Supabase 프로젝트 만들기 : 회원가입 설정

① 왼쪽 메뉴에서 Authentication을 선택합니다.

② Sign In / Providers 또는 Providers에서 Email을 엽니다.

③ 다음과 같이 설정합니다.

항목	실습 설정	의미
Allow new users to sign up	켜기 🟢	신규 회원가입 허용
Allow manual linking	끄기 ⚫	이번 실습에서 사용하지 않는 계정 연결 기능
Allow anonymous sign-ins	끄기 ⚫	익명 계정 로그인 사용 안 함
Confirm email	**끄기** ⚫	이메일 인증 없이 가입 후 로그인 가능

④ 저장합니다.

화면 구성이 다르면 Authentication 안에서 이메일 제공자 설정을 찾습니다.

진행 확인: 이번 실습에서는 가입 확인 이메일 없이 회원가입과 로그인을 진행합니다.

2. Codex로 DB 구성 SQL 만들기 : VSCode

① 컴퓨터에 madang 폴더를 만듭니다.
② VS Code에서 해당 폴더를 엽니다.
③ Codex를 엽니다.
④ 다음 프롬프트를 입력합니다.

```
지금부터 Supabase를 사용하는 온라인 서점을 단계별로 만들 것이다.

기술:
- 순수 HTML, CSS, JavaScript
- Supabase JavaScript SDK v2
- GitHub Pages 배포
- Node.js 서버, React, 빌드 도구는 사용하지 않는다.

기능:
- 도서 목록과 제목 검색
- 이메일·비밀번호 회원가입
- 로그인과 로그아웃
- 로그인한 회원의 도서 모의 구매
- 본인의 주문 내역 조회

실제 결제, 장바구니, 배송지, 재고 관리는 제외한다.

작업 규칙:
- 내가 요청한 단계만 진행한다.
- DB SQL은 파일로 작성한다.
- SQL을 Supabase에서 실행하는 것은 내가 직접 한다.
- 비밀번호를 저장하는 회원 테이블을 만들지 않는다.
- 회원 관리는 Supabase Auth를 사용한다.
- Secret key나 service_role 키를 웹 코드에 사용하지 않는다.
- 오류 해결을 위해 RLS를 끄지 않는다.
- 기존 데이터나 파일을 임의로 삭제하지 않는다.

지금은 코드를 만들지 말고 요구사항을 이해했는지만 간단히 설명해 줘.
```

2. Codex로 DB 구성 SQL 만들기 : VSCode

① 다음 프롬프트를 입력합니다.

온라인 서점의 Supabase DB 구성 SQL을 작성해 줘.

새 Supabase 프로젝트에서 한 번 실행하는
database.sql 파일로 만들어 줘.
아직 웹 파일은 만들지 마.

1. public.books 테이블
- id: 자동 생성 bigint 기본키
- title: 필수 문자열
- author: 필수 문자열
- price: 0 이상 정수, 원 단위

2. public.orders 테이블
- id: 자동 생성 bigint 기본키
- user_id: auth.users.id 참조, 필수
- book_id: public.books.id 참조, 필수
- book_title: 주문 당시 도서 제목, 필수
- quantity: 1~10 정수, 필수
- unit_price: 주문 당시 가격, 0 이상 정수, 필수
- created_at: 기본값이 현재 시각인 timestamptz, 필수
- user_id 인덱스 추가

3. 권한
- 두 테이블 모두 RLS 활성화
- books는 anon과 authenticated가 조회 가능
- 웹 사용자는 books 추가·수정·삭제 불가
- orders는 authenticated가 자기 주문만 조회 가능
- 본인 판단에는 auth.uid() 사용
- 웹 사용자의 orders 직접 INSERT/UPDATE/DELETE 불가
- 필요한 schema USAGE와 테이블 SELECT 권한 명시
- 기존 기본 권한이 남지 않도록 anon, authenticated의
  테이블 권한을 회수한 뒤 필요한 권한만 부여

4. 구매 함수
- 이름: public.purchase_book
- 매개변수: p_book_id bigint, p_quantity integer
- 반환값: 생성된 주문 id인 bigint
- auth.uid()로 구매자 결정
- 비로그인 사용자는 거절
- NULL 수량이나 1~10 범위 밖 수량은 거절
- 존재하지 않는 도서는 거절
- 제목과 가격은 books에서 읽어 orders에 저장
- user_id, 제목, 가격을 매개변수로 받지 않음
- SECURITY DEFINER 사용
- search_path를 빈 문자열로 고정
- 함수 내부 객체에 스키마 이름 명시
- PUBLIC, anon, authenticated의 함수 실행 권한 회수 후
  authenticated에만 EXECUTE 부여

5. 가상의 도서 샘플 6권 추가

6. 전체 작업을 하나의 트랜잭션으로 구성
7. DROP TABLE이나 기존 데이터 삭제 금지
8. 한국어 주석 작성

파일 생성 후 다음을 확인해 줘.
- 위 조건이 빠짐없이 반영되었는가?
- 같은 파일을 다시 실행하면 중복 생성 오류가 날 수 있는가?

마지막에 내가 복사해야 할 파일 이름을 알려줘.

생성결과 : database.sql를 열어 복사한다.

3. Supabase에 SQL 실행하기

① 왼쪽 메뉴에서 **SQL Editor**를 선택합니다.
② **New query** 또는 **+**를 선택합니다.
③ 가운데 SQL 입력 영역을 클릭합니다.
④ 복사한 SQL을 붙여 넣습니다.
⑤ **Run**을 선택합니다.
⑥ 화면 아래 결과 영역을 확인합니다.
⑦ 오류가 발생한 경우는 다음의 prompt를 사용

```
database.sql을 Supabase SQL Editor에 붙여 넣고 실행했는데
다음 오류가 발생했다.

[여기에 오류 메시지 전체 붙여 넣기]

현재 database.sql과 오류를 확인해 줘.
기존 테이블이나 데이터를 삭제하지 말고 해결해 줘.
RLS를 끄지 마.

실행 상태에 따라:
- 원본 SQL 전체를 다시 실행해야 하는지
- 별도 수정 SQL만 실행해야 하는지
명확히 구분해 줘.

수정 SQL이 필요하면 fix.sql 파일로 작성해 줘.
```

도서 데이터 확인

① 왼쪽 메뉴에서 **Table Editor**를 선택합니다.
② **public Schema** 를 확인 합니다.
③ **books**를 선택 합니다.
④ 도서 6권이 있는지 확인 합니다.
⑤ **orders**을 선택합니다.
⑥ 아직 주문이 없는 확인 합니다.

3. Supabase에 SQL 실행하기 : URL과 공개용 키 준비

① 프로젝트 상단의 **Connect**를 선택 합니다.

② **Project Settings** 의 API Keys, Data API 관련 확인후 복사 합니다.

- Api Keys : sb_publishable_.....
- Data API : https://프로젝트식별자.supabase.co (실제로는 더 길게 보이는데 왼쪽의 주소까지만 사용됩니다.)

4. Codex로 웹페이지 만들기

database.sql을 읽고 Supabase 도서 목록과 검색 화면을 만들어 줘.

이번 단계에서는 회원가입과 구매 기능을 만들지 마.

파일:
- index.html
- style.css
- config.js
- app.js

조건:
1. 순수 HTML/CSS/JavaScript 사용
2. Supabase JavaScript SDK v2를 CDN으로 불러오기
3. 브라우저 클라이언트 변수명은 sb 사용
4. SDK → config.js → app.js 순서로 실행되도록 구성
5. config.js에 Project URL과 Publishable key 자리표시자 작성
6. GitHub Pages에서 실행 가능한 상대 경로 사용
7. 한국어 화면
8. books에서 id, title, author, price 조회
9. 도서 제목, 저자, 원화 가격을 카드 형태로 표시
10. 제목 포함 검색은 ilike 사용
11. 빈 검색어는 전체 목록 조회
12. 로딩, 검색 결과 없음, 오류 상태 표시
13. DB 문자열은 textContent로 표시
14. 모든 Supabase 응답의 error 확인
15. IndexedDB나 임시 도서 배열로 대체하지 않기

완료 후 config.js에서 URL과 키를 붙여 넣을 위치를 알려줘.

① config.js를 엽니다.

② URL 자리표시자를 본인의 Project URL로 바꿉니다.

③ 키 자리표시자를 본인의 Publishable key로 바꿉니다.

④ 저장합니다.

⑤ index.html을 **Live Server**로 실행합니다.

확인: 브라우저에서 도서 6권이 보이는지 확인합니다.

검색과 실제 DB 연결 확인

[웹페이지]

① 도서 제목 일부를 검색합니다.

② 검색어를 지우고 전체 목록을 다시 확인합니다.

[Supabase]

③ Table Editor → books로 이동합니다.

④ 도서 한 권의 제목을 수정하고 저장합니다.

[웹페이지]

⑤ 웹페이지를 새로고침합니다.

⑥ 수정한 제목이 표시되는지 확인합니다.

확인: Supabase에서 변경한 내용이 웹에 보이면 DB 연결 성공입니다.

5. 회원가입 구매기능 연결하기

현재 온라인 서점에 Supabase Auth 회원 기능을 추가해 줘.
도서 목록과 검색 기능은 유지해 줘.

기능:
- 이메일·비밀번호 회원가입: signUp
- 이메일·비밀번호 로그인: signInWithPassword
- 로그아웃: signOut
- 로그인한 이메일 표시
- getSession으로 새로고침 후 로그인 상태 복원
- onAuthStateChange로 인증 상태 변화 반영

조건:
- 별도 회원 테이블을 만들지 않기
- 비밀번호를 자체 DB나 localStorage에 저장하지 않기
- 비밀번호 입력은 type=password
- 요청 중 버튼 비활성화
- 성공과 오류 메시지를 한국어로 표시
- 인증 상태 변경 콜백은 짧게 유지
- 콜백 내부에서 추가 Supabase 비동기 호출을 기다리지 않기

현재 Supabase에서 Confirm email을 꺼 두었다.
가입 후 session이 없으면 이메일 확인이 필요하다는 안내도 처리해 줘.

아직 구매 기능은 만들지 마.

[웹페이지]

① 회원가입합니다.

② 로그아웃합니다.

③ 가입한 계정으로 로그인합니다.

④ 새로고침합니다.

⑤ 로그인 상태가 유지되는지 확인합니다.

[Supabase]

⑥ **Authentication → Users**를 엽니다.

⑦ 본인의 가입 이메일이 있는지 확인합니다.

확인: 회원은 Authentication → Users, 도서는 Table Editor → books에서 확인합니다.

5. 회원가입 구매기능 연결하기

database.sql과 현재 코드를 읽고 모의 구매와 내 주문을 추가해 줘.

구매:
1. 도서 카드에 수량 입력과 구매 버튼 추가
2. 수량은 정수 1~10
3. 비로그인 사용자는 로그인 안내
4. 도서명, 수량, 예상 금액 확인 후 구매
5. orders에 직접 insert하지 않기
6. 구매는 다음 호출만 사용

```
sb.rpc('purchase_book', {
  p_book_id: 선택한도서번호,
  p_quantity: 수량
})
```

7. user_id, 가격, 제목을 구매 매개변수로 보내지 않기
8. 구매 처리 중 버튼 비활성화
9. 성공하면 "모의 구매가 완료되었습니다" 표시
10. 오류가 있으면 성공으로 처리하지 않기

내 주문:
- 로그인한 경우만 조회
- orders에서 본인의 주문 조회
- user_id 필터 사용, 최종 접근 제한은 RLS가 담당
- book_title, quantity, unit_price, created_at 표시
- 금액은 unit_price × quantity
- 최신 주문부터 표시
- 구매 성공 후 주문 목록 갱신
- 로그아웃 즉시 이전 주문 목록 비우기
- 회원 전환 시 이전 요청의 응답이 새 회원 화면에 표시되지 않도록 처리

실제 결제, 장바구니, 주문 수정·삭제는 추가하지 마.

[웹페이지]

① 로그인합니다.

② 도서 한 권을 선택합니다.

③ 수량 2를 입력합니다.

④ 구매 버튼을 누릅니다.

⑤ 내 주문 내역을 확인합니다.

[Supabase]

⑥ **Table Editor → orders**를 엽니다.

⑦ 주문이 저장되었는지 확인합니다.

확인: 웹페이지를 새로고침해도 주문이 남아 있어야 합니다.

다른 회원의 주문이 보이지 않는지 확인

순서	작업	정상 결과
①	A 계정으로 주문	A의 주문 표시
②	로그아웃	주문 목록 사라짐
③	B 계정으로 가입·로그인	A의 주문이 보이지 않음
④	B 계정으로 주문	B의 주문만 표시
⑤	A 계정으로 다시 로그인	A의 주문만 표시

6. Github Page로 배포 하기

[GitHub]

① GitHub에 로그인합니다.

② New repository를 선택합니다.

③ 다음과 같이 생성합니다.

항목	값
Repository name	madang
공개 설정	Public

④ Add file → Upload files를 선택합니다.

⑤ 다음 파일을 업로드합니다.

index.html

style.css

config.js

app.js

⑥ Commit changes를 선택합니다.

[GitHub]

① 저장소의 **Settings**를 선택합니다.

② 왼쪽의 **Pages**를 선택합니다.

③ 다음과 같이 설정합니다.

항목	값
Source	Deploy from a branch
Branch	main
Folder	/ (root)

④ Save를 누릅니다.

⑤ 배포 완료 후 표시되는 사이트 주소를 엽니다.

https://본인깃허브아이디.github.io/madang/

확인: 로컬 Live Server 주소가 아닌, GitHub Pages 주소로 접속했는지 확인합니다.

6. Github Page로 배포 하기: Supabase에 배포 주소 등록

[Supabase]

① **Authentication → URL Configuration**을 엽니다.

② Site URL에 본인의 배포 주소를 입력합니다.

https://본인깃허브아이디.github.io/madang/

③ Redirect URLs에 다음 주소를 추가합니다.

https://본인깃허브아이디.github.io/madang/

https://본인깃허브아이디.github.io/madang/index.html

④ 저장합니다.

이 주소는 이후 이메일 확인이나 비밀번호 재설정 등 인증 후 복귀할 때 사용합니다.

확인: 본인의 주소를 입력합니다.

배포한 서비스 최종 확인

[GitHub Pages 웹사이트]

① 도서 목록을 확인합니다.

② 제목을 검색합니다.

③ 회원가입 또는 로그인합니다.

④ 도서를 모의 구매합니다.

⑤ 내 주문을 확인합니다.

⑥ 새로고침 후 주문이 유지되는지 확인합니다.

⑦ 로그아웃합니다.

⑧ 다른 계정으로 로그인하여 주문이 분리되는지 확인합니다.

⑨ 휴대전화나 다른 PC에서 배포 주소를 열어봅니다.